

MANUAL TECNICO

CURSO

IPC1

CARNET

202401166

NOMBRE

JOSE ANTONIO

GARCIA ROCA

Índice

Introducción	3
Objetivos	3
Dirigido	3
Especificación Técnica.....	3
Requisitos de Hardware	3
Requisitos de Software.....	4
Lógica del Programa	5
Inicio de sesión	5
Menú Principal	5
Crear Usuario.....	6
Crear Cuenta	7
Buscar Cuenta	8
Depósitos	8
Retiros	9
Historial de Transacciones.....	9
Generación de Reportes	10
Generación de Bitácora	10
Datos Estudiante.....	11
Diagrama de Flujo.....	12

Introducción

El presente proyecto consiste en el desarrollo de una aplicación bancaria con interfaz gráfica, diseñada para gestionar usuarios, cuentas y transacciones financieras de manera eficiente. La aplicación sigue un flujo estructurado que inicia con un sistema de inicio de sesión, permitiendo al usuario acceder a diversas funcionalidades dentro de un menú principal. Entre las opciones principales se incluyen la creación de usuarios y cuentas, la búsqueda de cuentas asociadas a un usuario, la realización de depósitos y retiros, la consulta del historial de transacciones, la generación de reportes en formato PDF y la impresión de una bitácora de actividades en consola. Para garantizar la seguridad y consistencia de los datos, la aplicación impone restricciones como la validación de credenciales, la verificación de la unicidad del CUI para usuarios, la limitación del número de cuentas y usuarios permitidos, así como el control de transacciones por cuenta. Este sistema proporciona una experiencia intuitiva y estructurada para la gestión de operaciones bancarias, asegurando una administración ordenada y un registro detallado de cada acción realizada por los usuarios.

Objetivos

El objetivo de este proyecto es desarrollar una aplicación bancaria con interfaz gráfica que permita gestionar de manera eficiente y segura usuarios, cuentas y transacciones financieras, proporcionando una experiencia estructurada para la creación, administración y monitoreo de operaciones bancarias, con un enfoque en la validación de datos, control de transacciones y generación de reportes, garantizando la integridad y seguridad de la información.

Dirigido

Este manual está dirigido a quienes deseen comprender el desarrollo de esta aplicación, así como a programadores que busquen inspiración para aplicar métodos y funciones utilizados en este proyecto.

Especificación Técnica

Requisitos de Hardware

- Computadora de escritorio o portátil.
- Mínimo 4GB de Memoria RAM.
- 250GB de Disco Duro o superior.
- Procesador Intel Core i3 o superior.
- Resolución gráfica mínimo 1024 x 768 pixeles.

Requisitos de Software

- Sistema Operativo Windows7 o superior.
- Tener instalado Java Runtime Enviroment (JRE) versión 8.2 o superior.
- Tener instalado Java Development Kit (JDK) versión 8.2 o superior.
- Lenguaje de Programación JAVA.
- NetBeans IDE 8.2 o superior.
- Librería interna `java.util.Scanner`

Lógica del Programa

Inicio de sesión

```
public class LoginModel {
    public String user = "AdministradorIPC1E"; //AdministradorIPC1E
    public String password = "ipclEls2025"; // ipclEls2025

    public boolean logeo(String user, String password ){
        LocalDateTime ahora = LocalDateTime.now();
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");
        String fechaFormateada = ahora.format(formato);

        if (this.user.equals(user) && this.password.equals(password)) {
            Inicio.Inicio.getModeloBitacora().agregar("- INICIO DE SESION", fechaFormateada, "Sistema", "Inicio de Sesion",
            return true;
        }
        Inicio.Inicio.getModeloBitacora().agregar("- INICIO DE SESION", fechaFormateada, "Sistema", "Inicio de Sesion", "Err
        return false;
    }
}

public class UsuarioNombre{
    private static String nombreUsuario;

    public static String getNombreUsuario() {
        return nombreUsuario;
    }

    public static void setNombreUsuario(String nombreUsuario) {
        UsuarioNombre.nombreUsuario = nombreUsuario;
    }
}
```

Se encarga de verificar que las credenciales para iniciar sesión sean correctas, en caso de que sean incorrectas se muestra un mensaje de que las credenciales son incorrectas y en caso de que sean correctas se cierra la ventana del Inicio de sesión y se abre la ventana del Menú Principal, así como guardar el registro para posteriormente mostrarlo en la Bitácora.

Menú Principal

```
public class MenuModel {
    public String generarPdf(String cui, String usuario, List<Transacciones> transacciones, List<Cuentas> cuentas) {
        LocalDateTime ahora = LocalDateTime.now();
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("ddMMyyyyHHmmss");
        String fechaFormateada = ahora.format(formato);
        Document document = new Document();
        Document documentdepo = new Document();
        Document documentreti = new Document();
        List<Transacciones> transaccionesFiltradas = new ArrayList<>();

        //FiltrarTransaccionesPorCuentasPorUsuario()
        for(Cuentas cuentaUsuario: cuentas){
            if(cuentaUsuario.cui.equals(cui)){
                for(Transacciones usuarioTransaccion : transacciones){
                    if(usuarioTransaccion.cuentaGuid.equals(cuentaUsuario.numeroCuenta)){
                        transaccionesFiltradas.add(usuarioTransaccion);
                    }
                }
            }
        }

        //PDFTRANSACCIONESGENERALES
        try {
            PdfWriter.getInstance(document, new FileOutputStream("reportesTransacciones/" + "reporteTransacciones" + fecha
            document.open();

            document.add(new Paragraph("Reporte de Transacciones de " + usuario + " con CUI: " + cui));
            document.add(new Paragraph("\n\n"));
        }
    }
}
```

A través de esto es posible navegar entre cada ventana de nuestra aplicación, al presionar un botón se abre el menú de dicho botón y se cierra el menú principal, también es el encargado de generar los reportes de las cuentas, así como mostrar

los datos del estudiante y también la generación de la Bitácora, guarda también cada registro realizado en este menú para posteriormente mostrarlo en la Bitácora.

Crear Usuario

```
if(cui.length() != 13){
    Inicio.Inicio.getModeloBitacora().agregar("- REGISTRO DE USUARIO" ,fechaFormateada, "AdministradorIPCIE", "Regist
    return "Longitud del CUI invalida";
}

for (Usuario usuario : listaUsuarios){
    if(usuario.getCui().equals(cui)){
        Inicio.Inicio.getModeloBitacora().agregar("- REGISTRO DE USUARIO" ,fechaFormateada, "AdministradorIPCIE", "Re
        return "CUI repetido";
    }
}

if (listaUsuarios.size() > limite){
    Inicio.Inicio.getModeloBitacora().agregar("- REGISTRO DE USUARIO" ,fechaFormateada, "AdministradorIPCIE", "Regist
    return "No se permiten mas usuarios";
}

String mname = name.substring(0,1).toUpperCase() + name.substring(1,name.length());
String mlname = lname.substring(0,1).toUpperCase() + lname.substring(1,lname.length());

Usuario nuevoUsuario = new Usuario(mname, mlname, cui, cantidadCuentas);
listaUsuarios.add(nuevoUsuario);
Inicio.Inicio.getModeloBitacora().agregar("- REGISTRO DE USUARIO" ,fechaFormateada, "AdministradorIPCIE", "Registro c
return "Usuario agregado con exito";
}

public String cuentaNueva(int usuario){
    LocalDateTime ahora = LocalDateTime.now();
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");
    String fechaFormateada = ahora.format(formato);

    if(listaUsuarios.get(usuario).cantidadCuentas < 6){

    }

}

public class Usuario{
    public String name = "";
    public String lname = "";
    public String cui = "";
    public int cantidadCuentas = 0;

    public Usuario(String name, String lname, String cui, int cantidadCuentas){
        this.name = name;
        this.lname = lname;
        this.cui = cui;
        this.cantidadCuentas = cantidadCuentas;
    }
}
```

Se encarga de verificar la creación de usuarios, que tengan un CUI valido y demás verificaciones tales como que existan un máximo de 3 usuarios necesarios, así como verificar que cada usuario tenga un máximo de 6 cuentas, también se crea la clase Usuarios, la cual es la encargada de guardar las distintas características de los usuarios como su CUI, nombre, apellido y el numero de cuentas asociadas a este usuario, también envía el registro de creación para mostrarlo luego en la Bitácora.

Crear Cuenta

```
public String agregarCuenta(String cui, String name, String lname, String numeroCuenta, double saldo){
    LocalDateTime ahora = LocalDateTime.now();
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");
    String fechaFormateada = ahora.format(formato);

    Cuentas nuevaCuenta = new Cuentas(cui,name,lname,numeroCuenta,saldo,0);
    listaCuentas.add(nuevaCuenta);
    Inicio.getModeloBitacora().agregar("- CREACION DE CUENTA" ,fechaFormateada, "AdministradorIPCIE", "Creacion de cuenta");
    return "Cuenta Creada Correctamente";
}

public double depositar(double monto, double saldo, int idCuenta){
    for(Cuentas cuenta: listaCuentas){
        if(cuenta.getNumeroCuenta().equals(listaCuentas.get(idCuenta).getNumeroCuenta())){
            cuenta.setSaldo(saldo + monto);
            break;
        }
    }
    return saldo + monto;
}

public double retirar(double monto, double saldo, int idCuenta){
    for(Cuentas cuenta: listaCuentas){
        if(cuenta.getNumeroCuenta().equals(listaCuentas.get(idCuenta).getNumeroCuenta())){
            cuenta.setSaldo(saldo - monto);
            break;
        }
    }
}

}

public class Cuentas{
    public String cui;
    public String name;
    public String lname;
    public String numeroCuenta;
    public double saldo;
    public int numeroTransacciones;

    public Cuentas(String cui, String name, String lname, String numeroCuenta, double saldo, int numeroTransacciones){
        this.cui = cui;
        this.name = name;
        this.lname = lname;
        this.numeroCuenta = numeroCuenta;
        this.saldo = saldo;
        this.numeroTransacciones = numeroTransacciones;
    }

    public String getCui() {
        return cui;
    }

    public void setCui(String cui) {
```

Desde este código se crea la cuenta y se le asigna al usuario, también verifica si en cada cuenta se pueden realizar mas transacciones con un máximo de 25, también se crea la clase Cuenta en la cual se guardan los datos de cada cuenta, y también se manda a la Bitácora la respectiva creación de cuenta.

Buscar Cuenta

```
private void cargarCuentas() {
    LocalDateTime ahora = LocalDateTime.now();
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");
    String fechaFormateada = ahora.format(formato);

    limpiarTabla();
    int existe = 0;
    int i;
    List<Cuentas> cuentas = modeloCrearCuenta.getListaCuentas();
    List<Usuario> usuarios = modeloCrearUsuario.getListaUsuarios();
    String cui = this.vista.txtCui.getText();
    i = 0;
    for(Usuario comparar: usuarios){
        if(comparar.cui.equals(cui)){
            existe = 1;
            if(comparar.cantidadCuentas == 0){
                JOptionPane.showMessageDialog(null,"No se han asociado cuentas al usuario", "Buscar Cuentas", JOptionPane.ERROR_MESSAGE);
                Inicio.getModeloBitacora().agregar("- BUSQUEDA DE CUENTAS ASOCIADAS", fechaFormateada, "AdministradorIPC");
            }else{
                for(Cuentas pertenece : cuentas){
                    if (cui.equals(pertenece.cui)){
                        this.vista.tblCuentas.setValueAt(pertenece.numeroCuenta, i, 0);
                        i++;
                    }
                }
                Inicio.getModeloBitacora().agregar("- BUSQUEDA DE CUENTAS ASOCIADAS", fechaFormateada, "AdministradorIPC");
                break;
            }
        }
    }
    if(existe == 0){
        JOptionPane.showMessageDialog(null,"CUI inexistente", "Buscar Cuentas", JOptionPane.ERROR_MESSAGE);
        Inicio.getModeloBitacora().agregar("- BUSQUEDA DE CUENTAS ASOCIADAS", fechaFormateada, "AdministradorIPC", "Bus");
    }
}
```

A través del CUI, se obtiene que cuentas están asociadas a que usuario para posteriormente mostrarlas en una tabla.

Depósitos

```
public class DepositoModel{

    public String deposito(String monto, int cantidadTransacciones){
        LocalDateTime ahora = LocalDateTime.now();
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");
        String fechaFormateada = ahora.format(formato);

        if(monto.equals("")){
            Inicio.Inicio.getModeloBitacora().agregar("- DEPOSITO", fechaFormateada, "AdministradorIPC", "Deposito", "Error");
            return "No se ha ingresado un monto";
        }
        double montoD = Double.parseDouble(monto);
        if(montoD <= 0){
            Inicio.Inicio.getModeloBitacora().agregar("- DEPOSITO", fechaFormateada, "AdministradorIPC", "Deposito", "Error");
            return "El monto debe ser mayor a 0";
        }
        if (cantidadTransacciones < 25){
            return "Deposito realizado exitosamente";
        }
        Inicio.Inicio.getModeloBitacora().agregar("- DEPOSITO", fechaFormateada, "AdministradorIPC", "Deposito", "Error", " ");
        return "No se pueden realizar mas transacciones";
    }
}
```

Se encarga de verificar si es posible realizar un depósito a la cuenta destino en caso de que sea correcta se realiza el depósito y posteriormente se actualiza el dato de saldo en la respectiva cuenta a la que se le haya solicitado el depósito.

Retiros

```
public class RetiroModel {  
    public String retiro(String monto, Double saldo, int cantidadTransacciones){  
        LocalDateTime ahora = LocalDateTime.now();  
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("[yyyy-MM-dd HH:mm:ss]");  
        String fechaFormateada = ahora.format(formato);  
  
        if(saldo == 0){  
            Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Error", "E");  
            return "No se puede hacer el retiro, saldo = 0";  
        }  
  
        if(monto.equals("")){  
            Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Error", "E");  
            return "No se ha ingresado un monto";  
        }  
  
        double montoD = Double.parseDouble(monto);  
        if(montoD <= 0){  
            Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Error", "E");  
            return "El monto debe ser mayor a 0";  
        }  
  
        if(montoD > saldo){  
            Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Error", "E");  
            return "Saldo Insuficiente";  
        }  
  
        if (cantidadTransacciones < 25){  
            Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Exito", "E");  
            return "Retiro realizado exitosamente";  
        }  
  
        Inicio.Inicio.getModeloBitacora().agregar("- RETIRO", fechaFormateada, "AdministradorIPClE", "Retiro", "Error", "Limit");  
        return "No se pueden realizar mas transacciones";  
    }  
}
```

Se encarga de verificar que sea posible realizar un retiro a la cuenta destino, en casa de que se pueda se realiza el deposito y se actualiza el dato saldo de la respectiva clase Cuenta.

Historial de Transacciones

```
public int agregarTransaccion(String cuentaGuid, String fecha, String detalle, double debito, double credito, double saldo){  
    if(numeroTransacciones < 26){  
        numeroTransacciones = numeroTransacciones + 1;  
        int identificador = (int) ((Math.random()*9000)+1000);  
        Transacciones nuevaTransaccion = new Transacciones(cuentaGuid, identificador, fecha, detalle, debito, credito, saldo);  
        listaTransacciones.add(nuevaTransaccion);  
    }  
    System.out.println("");  
    return numeroTransacciones;  
}  
  
public class Transacciones{  
    public String cuentaGuid;  
    public int guid;  
    public String fecha;  
    public String detalle;  
    public double debito;  
    public double credito;  
    public double saldo;  
  
    public Transacciones(String cuentaGuid, int guid, String fecha, String detalle, double debito, double credito, double saldo){  
        this.cuentaGuid = cuentaGuid;  
        this.guid = guid;  
        this.fecha = fecha;  
        this.detalle = detalle;  
        this.debito = debito;  
        this.credito = credito;  
        this.saldo = saldo;  
    }  
}
```

Se encarga de crear la clase Transacciones, para posteriormente guardar cada transacción en esa clase, después de esto por medio de la solicitud del numero de cuenta se muestran en una tabla todas las transacciones realizadas por dicha cuenta.

Generación de Reportes

```
public class MenuModel {
    public String generarPdf(String cui, String usuario, List<Transacciones> transacciones, List<Cuentas> cuentas) {
        LocalDateTime ahora = LocalDateTime.now();
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("ddMMyyyyHHmmss");
        String fechaFormateada = ahora.format(formato);
        Document document = new Document();
        Document documentdepo = new Document();
        Document documentreti = new Document();
        List<Transacciones> transaccionesFiltradas = new ArrayList<>();

        //FiltrarTransaccionesPorCuentasPorUsuario()
        for (Cuentas cuentaUsuario: cuentas){
            if (cuentaUsuario.cui.equals(cui)){
                for (Transacciones usuarioTransaccion : transacciones){
                    if (usuarioTransaccion.cuentaGuid.equals(cuentaUsuario.numeroCuenta)){
                        transaccionesFiltradas.add(usuarioTransaccion);
                    }
                }
            }
        }

        //PDFTRANSACCIONESGENERALES
        try {

            PdfWriter.getInstance(document, new FileOutputStream("reportesTransacciones/" + "reporteTransacciones" + fechaF

            document.open();

            document.add(new Paragraph("Reporte de Transacciones de " + usuario + " con CUI: " + cui));
            document.add(new Paragraph("\n\n"));

            //CrearTabla
        }
    }
}
```

A través de este código se crea un PDF en el cual posteriormente se crea una tabla, dentro de esta tabla vamos guardando todos los objetos de la clase Transacciones, se generan todas las transacciones por medio del CUI del usuario, se genera un PDF donde se muestran todas las transacciones y otros dos donde se muestran únicamente los Depósitos y únicamente los Retiros.

Generación de Bitácora

```

public String agregar(String titulo ,String tiempo, String usuario, String accion, String resultado, String adicional){
    DatoBitacora dato = new DatoBitacora(titulo ,tiempo,usuario,accion,resultado,adicional);
    bitacora.add(dato);
    return "Lista Actualizada";
}

public class DatoBitacora{
    public String titulo;
    public String tiempo;
    public String usuario;
    public String accion;
    public String resultado;
    public String adicional;

    public DatoBitacora(String titulo ,String tiempo, String usuario, String accion, String resultado, String adicional) {
        this.titulo = titulo;
        this.tiempo = tiempo;
        this.usuario = usuario;
        this.accion = accion;
        this.resultado = resultado;
        this.adicional = adicional;
    }

    public String getTiempo() {
        return tiempo;
    }

    public void setTiempo(String tiempo) {
        this.tiempo = tiempo;
    }
}

```

Se crea la clase DatoBitacora, a través de la cual se guardan todos los datos realizados por la persona que utiliza la aplicación, toda esta información viene dada de los demás modelos, este modelo únicamente se encarga de almacenar todos los datos para cuando el usuario los solicite mostrarlos en consola.

Datos Estudiante

```

0      this.vista.lblUser.setText("Bienvenido " + LoginModel.UsuarioNombre.getNombreUsuario());
1
2      this.vista.mniDatos.addActionListener(new ActionListener() {
3          public void actionPerformed(ActionEvent e) {
4              LocalDateTime ahora = LocalDateTime.now();
5              DateTimeFormatter formato = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
6              String fechaFormateada = ahora.format(formato);
7
8              JOptionPane.showMessageDialog(null,"José García \n202401166 \nIPC1 'E',"Datos Estudiante",JOptionPane.INFORMATION_MESSAGE);
9              Inicio.getModeloBitacora().agregar("- DATOS DEL ESTUDIANTE" ,fechaFormateada, "AdministradorIPC1E", "Consulta");
10         }
11     }

```

Muestra una ventana flotante con los datos del estudiante.

Diagrama de Flujo

